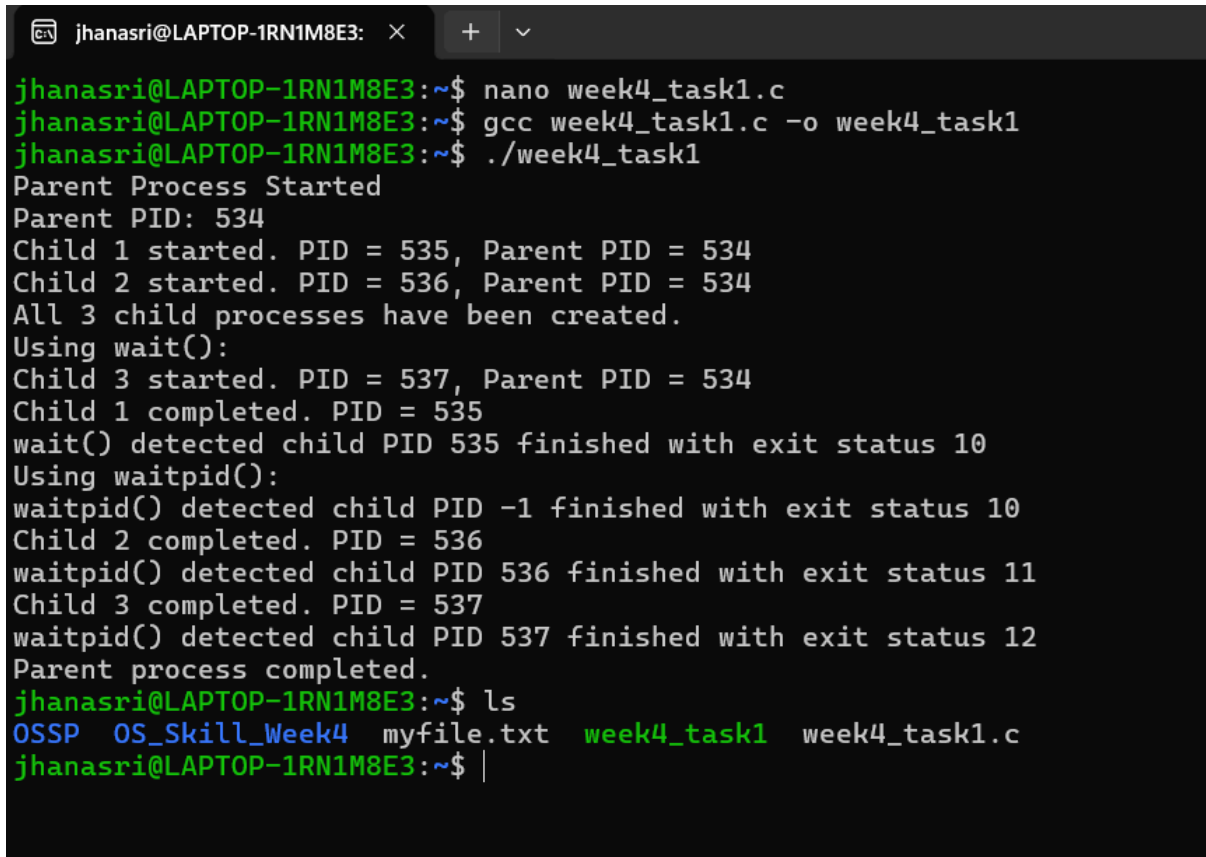


Task 3:

Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.



```
jhanasri@LAPTOP-1RN1M8E3: ~$ nano week4_task1.c
jhanasri@LAPTOP-1RN1M8E3: ~$ gcc week4_task1.c -o week4_task1
jhanasri@LAPTOP-1RN1M8E3: ~$ ./week4_task1
Parent Process Started
Parent PID: 534
Child 1 started. PID = 535, Parent PID = 534
Child 2 started. PID = 536, Parent PID = 534
All 3 child processes have been created.
Using wait():
Child 3 started. PID = 537, Parent PID = 534
Child 1 completed. PID = 535
wait() detected child PID 535 finished with exit status 10
Using waitpid():
waitpid() detected child PID -1 finished with exit status 10
Child 2 completed. PID = 536
waitpid() detected child PID 536 finished with exit status 11
Child 3 completed. PID = 537
waitpid() detected child PID 537 finished with exit status 12
Parent process completed.
jhanasri@LAPTOP-1RN1M8E3: ~$ ls
OSSP  OS_Skill_Week4  myfile.txt  week4_task1  week4_task1.c
jhanasri@LAPTOP-1RN1M8E3: ~$ |
```

Step 1: Open Ubuntu Terminal

Press:

Ctrl + Alt + T

Step 2: Create a C file

Type:

nano week4_task1.c

Step 3: Enter the C program

Paste your Task 1 C program into the editor.

If you need the complete program, use:

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>


int main() {
    pid_t child[3];
    int status;
    int i;


    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());


    for (i = 0; i < 3; i++) {
        child[i] = fork();


        if (child[i] < 0) {
            perror("fork failed");
            exit(1);
        }


        if (child[i] == 0) {
            printf("Child %d started. PID = %d, Parent PID = %d\n",
                i + 1, getpid(), getppid());


            sleep((i + 1) * 2);


            printf("Child %d completed. PID = %d\n", i + 1, getpid());


            exit(i + 10);
        }
    }
}
```

```

    }
}

printf("All 3 child processes have been created.\n");

printf("Using wait():\n");

pid_t pid = wait(&status);

if (WIFEXITED(status)) {
    printf("wait() detected child PID %d finished with exit status %d\n",
        pid, WEXITSTATUS(status));
}

printf("Using waitpid():\n");

for (i = 0; i < 3; i++) {
    pid = waitpid(child[i], &status, 0);

    if (WIFEXITED(status)) {
        printf("waitpid() detected child PID %d finished with exit status %d\n",
            pid, WEXITSTATUS(status));
    }
}

printf("Parent process completed.\n");

return 0;
}

```

Step 4: Save the file

In **nano**:

Ctrl + O

Press **Enter**.

Then:

Ctrl + X

Step 5: Compile the program

Type:

```
gcc week4_task1.c -o week4_task1
```

If there is **no error**, compilation is successful.

Step 6: Run the program

Type:

```
./week4_task1
```

Step 7: Observe the output

You should see something similar to:

Parent Process Started

Parent PID: 4521

Child 1 started. PID = 4522, Parent PID = 4521

Child 2 started. PID = 4523, Parent PID = 4521

Child 3 started. PID = 4524, Parent PID = 4521

All 3 child processes have been created.

Using wait():

Child 1 completed. PID = 4522

wait() detected child PID 4522 finished with exit status 10

Using waitpid():

Child 2 completed. PID = 4523

Child 3 completed. PID = 4524

waitpid() detected child PID 4523 finished with exit status 11

waitpid() detected child PID 4524 finished with exit status 12

Parent process completed.

Your practical specifically demonstrates creating **three children**, then using `wait()` and `waitpid()` to synchronize their completion.

Step 8: If you want to check the file

ls

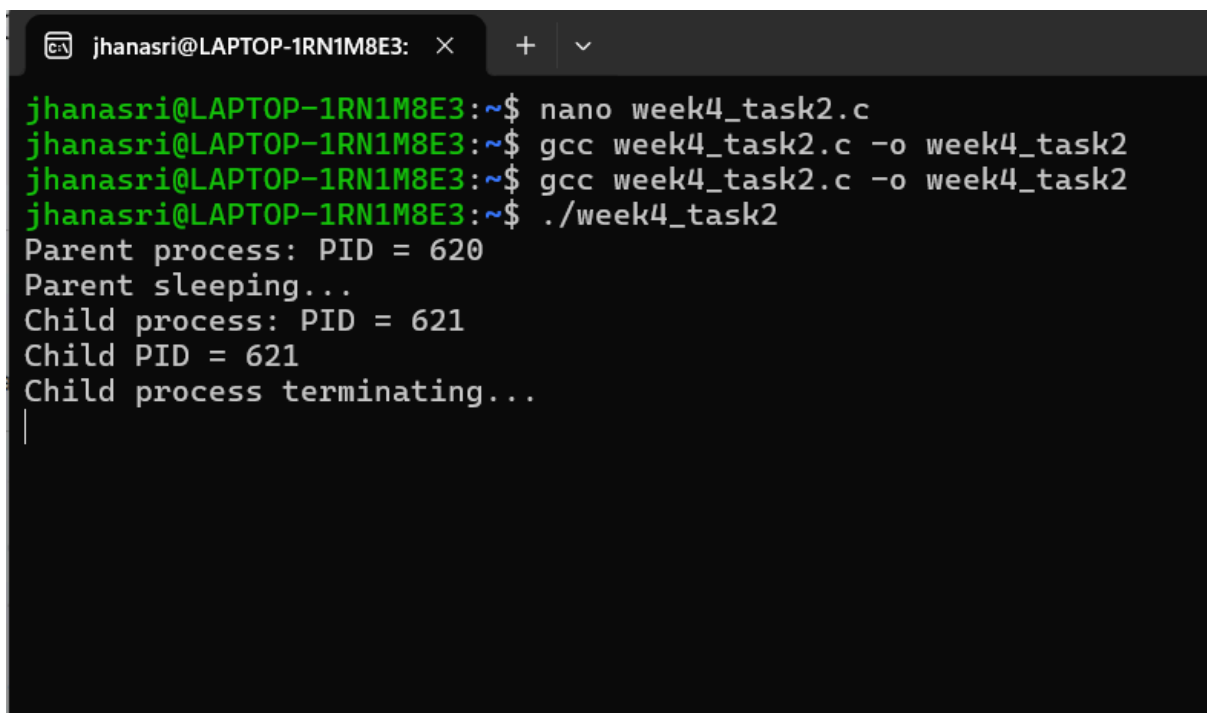
You should see:

week4_task1.c

Week4_task1

Task 4:

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.



```
jhanasri@LAPTOP-1RN1M8E3: ~$ nano week4_task2.c
jhanasri@LAPTOP-1RN1M8E3: ~$ gcc week4_task2.c -o week4_task2
jhanasri@LAPTOP-1RN1M8E3: ~$ gcc week4_task2.c -o week4_task2
jhanasri@LAPTOP-1RN1M8E3: ~$ ./week4_task2
Parent process: PID = 620
Parent sleeping...
Child process: PID = 621
Child PID = 621
Child process terminating...
|
```

```

4 S    100      209        1 0 80 0 - 55137 -      ?      00:00:00 rsyslogd
4 S    988      257       130 0 80 0 - 5933 -      ?      00:00:00 chronyd
4 S     0      261        1 0 80 0 - 30865 -      ?      00:00:00 unattended-up
1 S    988      271       257 0 80 0 - 3024 -      ?      00:00:00 chronyd
4 S     0      286        1 0 80 0 - 1373 -      tty1    00:00:00 agetty
5 S     0      345        2 0 80 0 - 799 -      ?      00:00:00 SessionLeader
5 S     0      346       345 0 80 0 - 799 -      ?      00:00:00 Relay(347)
4 S   1000      347       346 0 80 0 - 1567 poll_s pts/0    00:00:00 bash
4 S     0      348        2 0 80 0 - 2162 -      ?      00:00:00 login
4 S   1000      395        1 0 80 0 - 5687 -      ?      00:00:00 systemd
5 S   1000      399       395 0 80 0 - 5729 -      ?      00:00:00 (sd-pam)
4 S   1000      426       348 0 80 0 - 1534 poll_s pts/1    00:00:00 bash
5 S     0      552        2 0 80 0 - 799 -      ?      00:00:00 SessionLeader
5 S     0      553       552 0 80 0 - 799 -      ?      00:00:00 Relay(554)
4 S   1000      554       553 0 80 0 - 1567 poll_s pts/2    00:00:00 bash
5 S     0      622        2 0 80 0 - 799 -      ?      00:00:00 SessionLeader
5 S     0      623       622 0 80 0 - 799 -      ?      00:00:00 Relay(624)
4 S   1000      624       623 0 80 0 - 1567 do_wai pts/3    00:00:00 bash
0 R   1000      654       624 0 80 0 - 1798 -      pts/3    00:00:00 ps
jhanasri@LAPTOP-1RN1M8E3:~$ nano week4_task2.c
jhanasri@LAPTOP-1RN1M8E3:~$ nano week4_task2.c
jhanasri@LAPTOP-1RN1M8E3:~$ gcc week4_task2.c -o week4_task2
jhanasri@LAPTOP-1RN1M8E3:~$ ./week4_task2
Parent process: PID = 738
Child process: PID = 739
Child PID = 739
Child process terminating...
Child PID = 739 terminated with exit status 0
Parent terminating...
jhanasri@LAPTOP-1RN1M8E3:~$ |

```

Step 1: Open Ubuntu Terminal

Press:

Ctrl + Alt + T

Step 2: Create the C file

nano week4_task2.c

Step 3: Enter the Zombie Program

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
int main() {
```

```
    pid_t pid;
```

```
    pid = fork();
```

```

if (pid < 0) {
    perror("fork failed");
    exit(1);
}

if (pid == 0) {
    printf("Child process: PID = %d\n", getpid());
    printf("Child PID = %d\n", getpid());

    printf("Child process terminating...\n");
    exit(0);
}
else {
    printf("Parent process: PID = %d\n", getpid());
    printf("Parent sleeping...\n");

    sleep(20);

    printf("Parent terminating...\n");
}

return 0;
}

```

This follows the first part of your practical: the child terminates while the parent remains alive/sleeping, creating the opportunity to observe the zombie.

Step 4: Save the program

Press:

Ctrl + O

Press **Enter**.

Then press:

Ctrl + X

Step 5: Compile the program

```
gcc week4_task2.c -o week4_task2
```

If no error appears, compilation is successful.

Step 6: Run the program

```
./week4_task2
```

You will get something like:

Parent process: PID = 158

Parent sleeping...

Child process: PID = 159

Child PID = 159

Child process terminating...

Step 7: Quickly open another terminal

While the **parent is sleeping**, open another terminal:

Ctrl + Alt + T

Step 8: Check the process table

Run:

```
ps -el
```

Step 9: Find your program

Look for:

```
week4_task2
```

The child should appear with **Z** in the status column.

Example from your practical:

```
0 Z 1000 159 158 ... week4_T2
```

Here, **Z = Zombie process**.

Step 10: Go back to the first terminal

Wait for:

Parent terminating...

The parent then finishes.

Now Eliminate the Zombie

Step 11: Open the C file again

nano week4_task2.c

Step 12: Add `waitpid()`

First add this header:

```
#include <sys/wait.h>
```

Then, inside the **parent section**, add:

```
int status;
```

```
waitpid(pid, &status, 0);
```

So the parent part becomes:

```
else {  
    printf("Parent process: PID = %d\n", getpid());  
  
    waitpid(pid, &status, 0);  
  
    printf("Parent terminating...\n");  
}
```

Step 13: Save

Ctrl + O

Press **Enter**.

Then:

Ctrl + X

Step 14: Compile again

```
gcc week4_task2.c -o week4_task2
```

Step 15: Run again

```
./week4_task2
```

Step 16: Check the process table

While the program is running, open another terminal:

```
ps -el
```

Step 17: Check for Z

Previously, you saw:

```
Z
```

Now the parent uses `waitpid()` to collect the child's termination status, so the child should **not remain as a zombie**.

Step 18: Finish

The parent prints:

Parent terminating...

and the program ends.